

PONDICHERRY UNIVERSITY



SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

Name : ABDULLA AZIYAN C K

Register Number : 23384101

Semester : VI

Subject : DISTRIBUTED SYSTEMS

Subject code : CSCS308

TABLE OF CONTENTS

EX. NO.	TITLE
1	Socket Programming – Client-Server Communication using Python
2	RMI / Simulate RPC in Python
3	Message Passing Between Two Processes using Python
4	Lamport Logical Clock Algorithm using Python
5	Distributed Sum of N Numbers using Python
6	Distributed Average / Mean Calculation using Python
7	Multi-threaded Server Handling Multiple Clients using Python
8	Distributed Chat Application using Sockets using Python
9	Bully Leader Election Algorithm using Python
10	Ring-based Leader Election Algorithm using Python

AIM:

To implement a two-way chat communication between a server and a client using Python's socket programming.

CODE:**SERVER.PY**

```
import socket

server = socket.socket()

server.bind(('localhost', 5000))

server.listen(1)

print("Waiting for connection...")

conn, addr = server.accept()

print(f'Connected: {addr}\nType 'bye' to exit\n")

while True:

    msg = conn.recv(1024).decode()

    print("Client:", msg)

    if msg.lower() == 'bye': break

    reply = input("Server: ")

    conn.send(reply.encode())

    if reply.lower() == 'bye': break

conn.close()

server.close()
```

CLIENT.PY

```
import socket

client = socket.socket()

client.connect(('localhost', 5000))

print("Connected! Type 'bye' to exit\n")

while True:

    msg = input("Client: ")

    client.send(msg.encode())

    if msg.lower() == 'bye': break

    reply = client.recv(1024).decode()

    print("Server:", reply)

    if reply.lower() == 'bye': break

client.close()
```

OUTPUT:

A screenshot of the Visual Studio Code editor interface. The top bar shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (selected), and PORTS. On the right side of the top bar are icons for running Python code, a dropdown menu, a window icon, a trash icon, and other standard IDE controls. Below the top bar, there are two terminal windows. The left terminal window displays a netcat listener on port 55734, connected to 127.0.0.1. It shows a client sending "Hii", the server responding "hello", the client sending "how are you", the server responding "fine", and finally the client saying "bye". The right terminal window shows the output of a Python script named "communication/client.py", which mirrors the same conversation between a client and a server.

RESULT

Thus, the simple client-server communication program using socket programming in Python was implemented successfully and messages were exchanged between server and client.

AIM:-

To simulate Remote Procedure Call (RPC) in Python using socket programming and perform arithmetic operations between client and server.

CODE:

```
import socket

server = socket.socket()

server.bind(('localhost', 5000))

server.listen(1)

print("RPC Server Waiting...")

while True:

    conn, addr = server.accept()

    print("Connected:", addr)

    data = conn.recv(1024).decode()

    if data.lower() == "bye":

        print("Server Closed")

        conn.close()

        break

    parts = data.split()

    op = parts[0]

    a = int(parts[1])

    b = int(parts[2])

    if op == "add":

        result = a + b

    elif op == "sub":

        result = a - b
```

```
elif op == "mul":
    result = a * b
elif op == "div":
    result = a / b
else:
    result = "Invalid Operation"
conn.send(str(result).encode())
conn.close()
server.close()
CLIENT.PY
import socket
while True:
    client = socket.socket()
    client.connect(('localhost', 5000))
    op = input("Enter operation (add/sub/mul/div) or bye: ")
    if op.lower() == "bye":
        client.send("bye".encode())
        client.close()
        break
    a = input("Enter first number: ")
    b = input("Enter second number: ")
    msg = op + " " + a + " " + b
    client.send(msg.encode())
    result = client.recv(1024).decode()
    print("Result:", result)
    client.close()
```

OUTPUT:

```
/Users/R.BAVADHARANI/OneDrive/Desktop/Distributed System Lab Experiments/RPC-PY THON/client.py"
THON/server.py"
RPC Server Waiting...
Connected: ('127.0.0.1', 58781)
Connected: ('127.0.0.1', 58784)
Enter operation (add/sub/mul/div) or bye: add
Enter first number: 50
Enter second number: 45
Result: 95
Enter operation (add/sub/mul/div) or bye: 
```

RESULT:-

Thus, the program to simulate Remote Procedure Call (RPC) in Python was implemented successfully and the arithmetic operation was executed remotely.

AIM:

To demonstrate message passing between two processes using Python multiprocessing and queue.

CODE:

```
from multiprocessing import Process, Queue

def sender(q, msg):
    q.put(msg)
    print("Message Sent")

def receiver(q):
    msg = q.get()
    print("Message Received:", msg)

if __name__ == "__main__":
    q = Queue()
    msg = input("Enter message: ")
    p1 = Process(target=sender, args=(q, msg))
    p2 = Process(target=receiver, args=(q,))
    p1.start()
    p1.join()
    p2.start()
    p2.join()
```


OUTPUT:

```
PS C:\Users\R.BAVADHARANI\OneDrive\Desktop\Distributed System Lab Experiments> & C:/Users/R.BAVADHARANI/AppData/Local/Programs/Python/Python39-32/Scripts/python.exe C:/Users/R.BAVADHARANI/OneDrive/Desktop/Distributed System Lab Experiments/Message-Passes-two-processes/process.py
Enter message: HELLOO
Message Sent
Message Received: HELLOO
PS C:\Users\R.BAVADHARANI\OneDrive\Desktop\Distributed System Lab Experiments> █
```

RESULT:

Thus, the program to demonstrate message passing between two processes using Python was implemented successfully.

AIM:

To simulate Lamport Logical Clock Algorithm for ordering events in distributed systems using Python.

CODE:

```
p1 = int(input("Enter initial clock of P1: "))
p2 = int(input("Enter initial clock of P2: "))
p1 += 1
print("P1 Internal Event:", p1)
p1 += 1
msg = p1
print("P1 Sends Message at:", msg)
p2 = max(p2, msg) + 1
print("P2 Receives Message at:", p2)
```

OUTPUT:

```
PS C:\Users\R.BAVADHARANI\OneDrive\Desktop\Distributed System Lab Experiments> & C:/Users/R.BAVADHARANI/AppData/Local/Programs/Python/Python38-32/Python.exe C:/Users/R.BAVADHARANI/OneDrive/Desktop/Distributed System Lab Experiments/Lamport-Clock/lamport.py
Enter initial clock of P1: 0
Enter initial clock of P2: 0
P1 Internal Event: 1
P1 Sends Message at: 2
P2 Receives Message at: 3
```

RESULT:

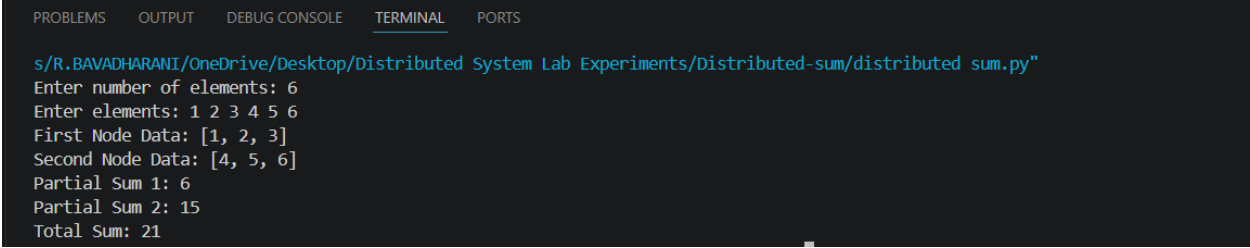
Thus, the Lamport Logical Clock Algorithm was simulated successfully and event ordering was demonstrated.

AIM:

To implement distributed sum of N numbers by dividing data among nodes and calculating the total sum using Python.

CODE:

```
n = int(input("Enter number of elements: "))
nums = list(map(int, input("Enter elements: ").split()))
mid = n // 2
part1 = nums[:mid]
part2 = nums[mid:]
sum1 = sum(part1)
sum2 = sum(part2)
total = sum1 + sum2
print("First Node Data:", part1)
print("Second Node Data:", part2)
print("Partial Sum 1:", sum1)
print("Partial Sum 2:", sum2)
print("Total Sum:", total)
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

s/R.BAVADHARANI/OneDrive/Desktop/Distributed System Lab Experiments/Distributed-sum/distributed sum.py"
Enter number of elements: 6
Enter elements: 1 2 3 4 5 6
First Node Data: [1, 2, 3]
Second Node Data: [4, 5, 6]
Partial Sum 1: 6
Partial Sum 2: 15
Total Sum: 21
```

RESULT:

Thus, the distributed sum of N numbers was implemented successfully and the total sum was obtained.

AIM: To calculate the average of numbers by dividing data into distributed parts and combining results.

CODE:

```
def part_average(data):  
    return sum(data), len(data)  
  
n = int(input("Enter number of elements: "))  
data = list(map(int, input("Enter elements: ").split()))  
parts = int(input("Enter number of parts: "))  
size = n // parts  
total_sum = 0  
total_count = 0  
for i in range(parts):  
    part = data[i*size:(i+1)*size]  
    s, c = part_average(part)  
    total_sum += s  
    total_count += c  
avg = total_sum / total_count  
print("Distributed Average:", avg)
```

OUTPUT:

```
✓ TERMINAL
PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python38-32/Scripts/python.exe C:/Users/Asus/Desktop/Distributed system lab/Simple Compound Interest/Distributed Average.py
Enter number of elements: 5
Enter elements: 10 15 20 25 30
Enter number of parts: 4
Distributed Average: 17.5
PS C:\Users\Asus\Desktop\Distributed system lab> |
```

RESULT: The average was successfully calculated using distributed partitions.

AIM: To develop a multi-threaded server that can handle multiple client requests simultaneously using sockets in Python.

CODE:

SERVER.PY

```
import socket

import threading

def handle_client(conn, addr):
    print("Connected:", addr)
    msg = conn.recv(1024).decode()
    print("Client:", msg)
    conn.send("Message received".encode())
    conn.close()

server = socket.socket()
server.bind(("localhost", 5000))
server.listen(5)
print("Server started...")
while True:
    conn, addr = server.accept()
    thread = threading.Thread(target=handle_client, args=(conn, addr))
    thread.start()
```

CLIENT.PY

```
import socket

client = socket.socket()

client.connect(("localhost", 5000))

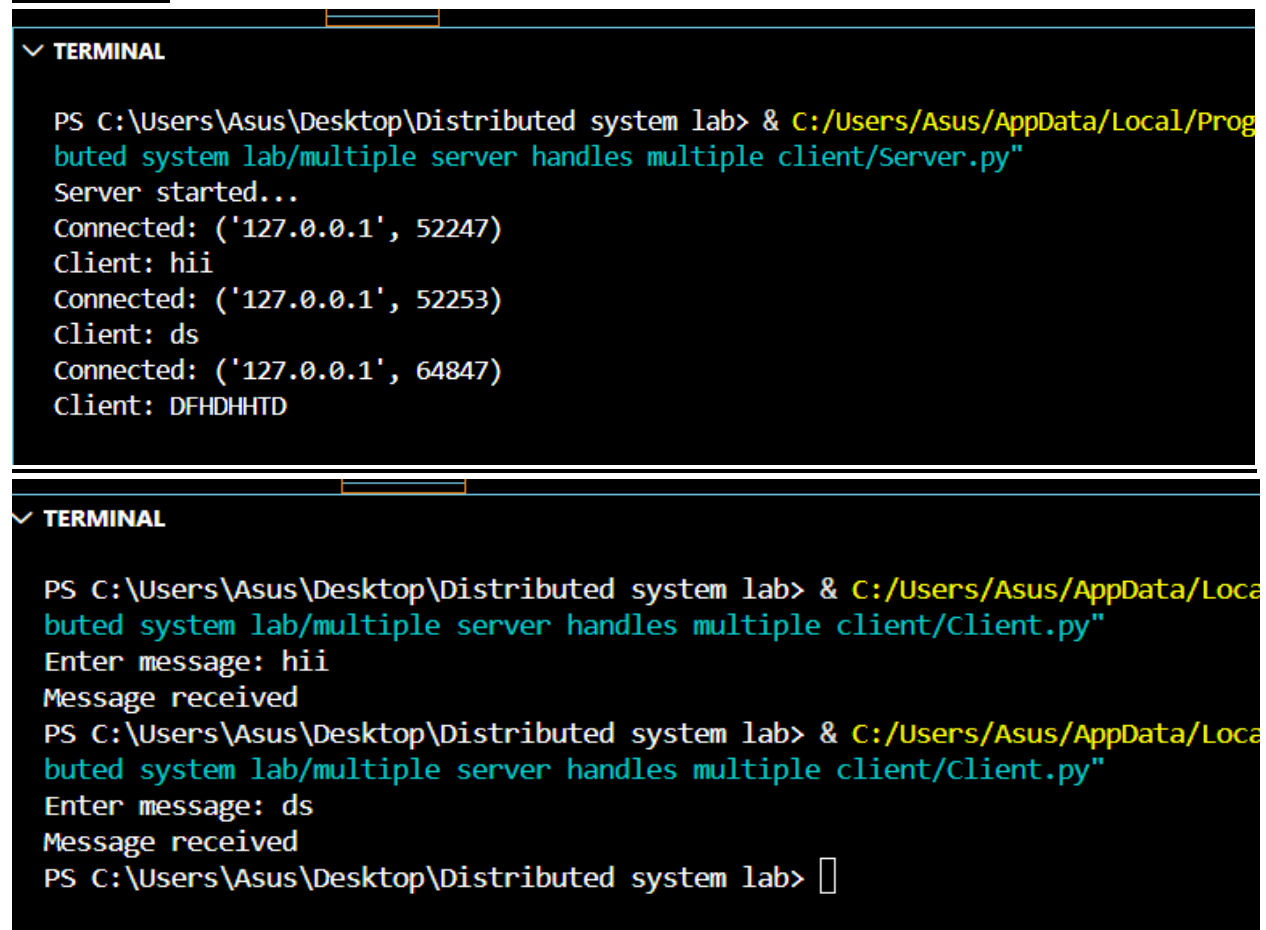
msg = input("Enter message: ")

client.send(msg.encode())

print(client.recv(1024).decode())

client.close()
```

OUTPUT:



The image shows two screenshots of a Windows terminal window. The first screenshot shows the execution of a server script named 'multiple server handles multiple client/Server.py'. The terminal output indicates the server started successfully and then received three consecutive connections from the IP address '127.0.0.1' at ports 52247, 52253, and 64847. The messages received from the clients were 'hii', 'ds', and 'DFHDHHTD' respectively. The second screenshot shows the execution of a client script named 'multiple server handles multiple client/Client.py'. The terminal output shows the client entering the message 'hii', which was received by the server. Then, the client enters the message 'ds', which was also received by the server. The terminal prompt is shown at the end of the second screenshot.

```
PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python38-64/Scripts/python.exe C:/Users/Asus/Desktop/Distributed system lab/multiple server handles multiple client/Server.py
Server started...
Connected: ('127.0.0.1', 52247)
Client: hii
Connected: ('127.0.0.1', 52253)
Client: ds
Connected: ('127.0.0.1', 64847)
Client: DFHDHHTD

PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python38-64/Scripts/python.exe C:/Users/Asus/Desktop/Distributed system lab/multiple server handles multiple client/Client.py
Enter message: hii
Message received
PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python38-64/Scripts/python.exe C:/Users/Asus/Desktop/Distributed system lab/multiple server handles multiple client/Client.py
Enter message: ds
Message received
PS C:\Users\Asus\Desktop\Distributed system lab> 
```

RESULT:

The multi-threaded server was successfully implemented, and it handled multiple clients concurrently using threads.

AIM: To implement a two-way communication chat system using sockets.

CODE:

SERVER.PY

```
import socket
server = socket.socket()
server.bind(("localhost", 6000))
server.listen(1)
conn, addr = server.accept()
print("Connected:", addr)
while True:
    msg = conn.recv(1024).decode()
    print("Client:", msg)
    reply = input("You: ")
    conn.send(reply.encode())
```

CLIENT.PY

```
import socket
client = socket.socket()
client.connect(("localhost", 6000))
while True:
    msg = input("You: ")
    client.send(msg.encode())
    reply = client.recv(1024).decode()
    print("Server:", reply)
```


OUTPUT:

```
✓ TERMINAL

PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python38-32/Python.exe C:\Users\Asus\Desktop\Distributed system lab/DISTRIBUTED CHAT APPLICATION/Server.py"
Connected: ('127.0.0.1', 64112)
Client: hi
You: hello
█
```

```
✓ TERMINAL

PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python38-32/Python.exe C:\Users\Asus\Desktop\Distributed system lab/DISTRIBUTED CHAT APPLICATION/client.py"
You: hi
Server: hello
You: █
```

RESULT:

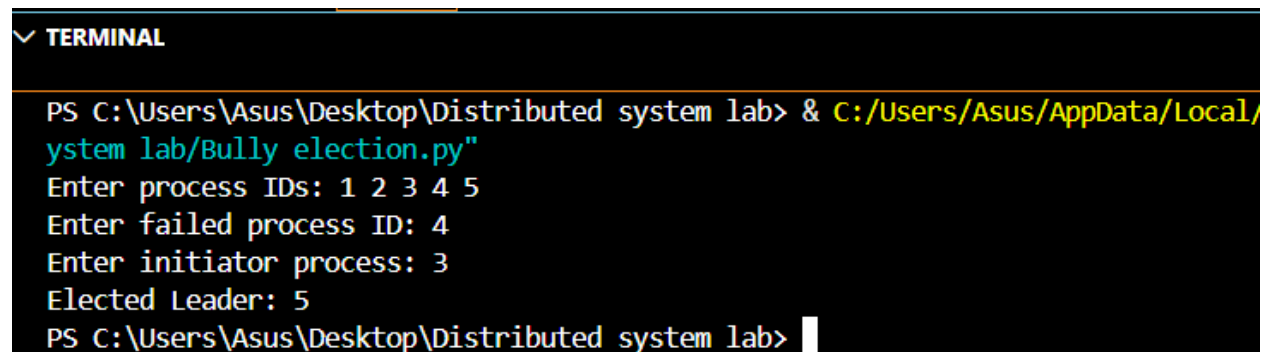
Two-way communication between client and server was achieved successfully.

AIM: To elect a leader using Bully Algorithm where the highest process ID becomes leader.

CODE:

```
processes = list(map(int, input("Enter process IDs: ").split()))
failed = int(input("Enter failed process ID: "))
active = [p for p in processes if p != failed]
initiator = int(input("Enter initiator process: "))
higher = [p for p in active if p > initiator]
if not higher:
    leader = initiator
else:
    leader = max(active)
print("Elected Leader:", leader)
```

OUTPUT:



```
✓ TERMINAL
PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/
system lab/Bully election.py"
Enter process IDs: 1 2 3 4 5
Enter failed process ID: 4
Enter initiator process: 3
Elected Leader: 5
PS C:\Users\Asus\Desktop\Distributed system lab> █
```

RESULT:

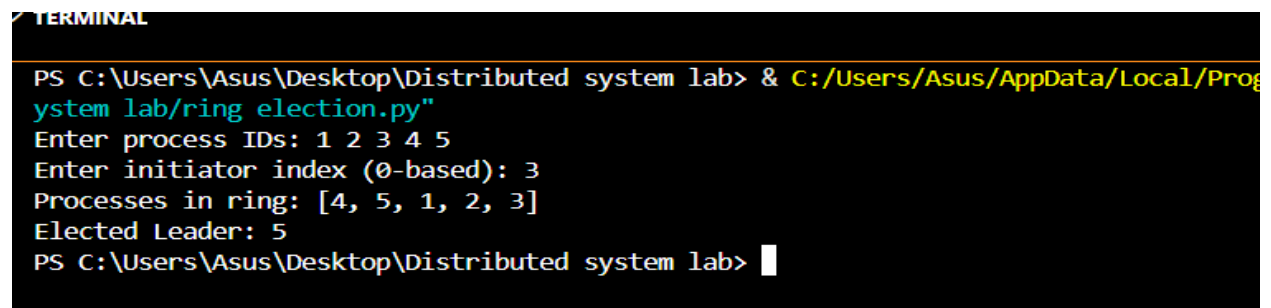
Leader election was successfully performed using Bully Algorithm.

AIM: To elect a leader in a ring topology where processes pass messages in order.

CODE:

```
processes = list(map(int, input("Enter process IDs: ").split()))
n = len(processes)
initiator = int(input("Enter initiator index (0-based): "))
msg = []
i = initiator
while True:
    msg.append(processes[i])
    i = (i + 1) % n
    if i == initiator:
        break
leader = max(msg)
print("Processes in ring:", msg)
print("Elected Leader:", leader)
```

OUTPUT:



```
PS C:\Users\Asus\Desktop\Distributed system lab> & C:/Users/Asus/AppData/Local/Programs/Python/Python39-64/Scripts/python C:\Users\Asus\Desktop\Distributed system lab\ring election.py
Enter process IDs: 1 2 3 4 5
Enter initiator index (0-based): 3
Processes in ring: [4, 5, 1, 2, 3]
Elected Leader: 5
PS C:\Users\Asus\Desktop\Distributed system lab>
```

RESULT:

Leader was successfully elected using ring-based algorithm.